Primera Práctica de Programación

Estudiante: Fabián Vargas Hidalgo

Universidad Cenfotec

Curso: Programación Orientada a Objetos

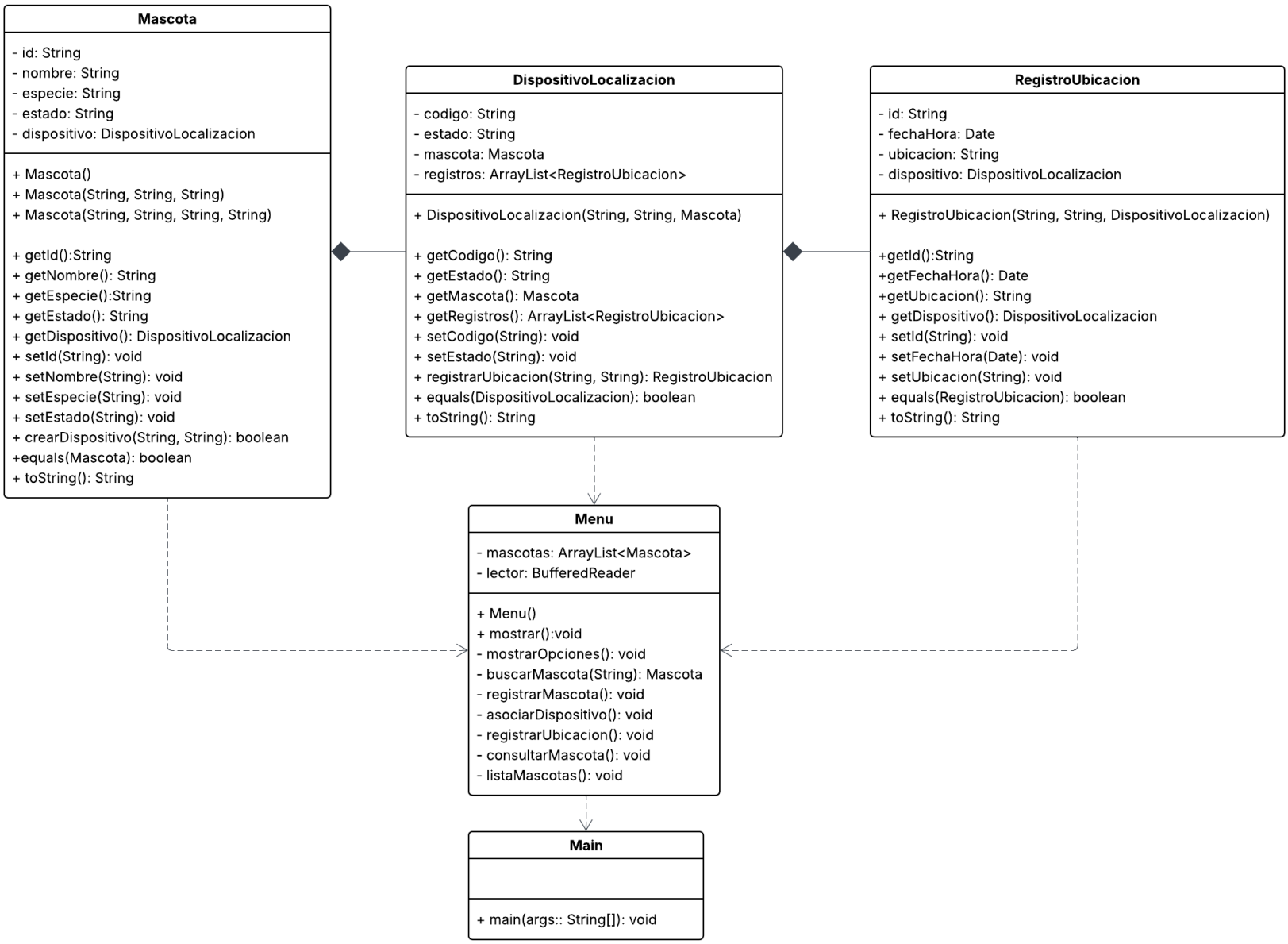
Código: SOFT-04

Profesor: Romario Salas Cerdas

Fecha de entrega: 08 de marzo del 2026

Link github: https://github.com/FabianVargasH/PrimeraPracticaProgramada-POO

Diagrama UML



Explicación del modelo y sus relaciones

Visión General del Modelo

El sistema implementa tres clases principales que representan el dominio del problema:

Clase	Propósito
Mascota	Representa el animal que será monitoreado
DispositivoLocalizacion	Representa el collar inteligente asociado a la mascota
RegistroUbicacion	Representa cada captura de ubicación realizada por el dispositivo

Estas clases junto con el menú y el main trabajan en conjunto para cumplir con los casos de uso: Registro de mascotas, asociación de dispositivos, registro de ubicaciones y consulta de información.

Analisis de las Relaciones según Consigna

La consigna establece claramente:

1. Una mascota se asocia con un único dispositivo de localización.
2. Cada dispositivo pertenece a una única mascota.
3. Un dispositivo puede estar asociado con cero o más registros de ubicación.
4. Cada registro de ubicación pertenece a un único dispositivo.
5. El dispositivo no puede existir de forma independiente.

Estos puntos definen las cardinalidades y el ciclo de vida de los objetos, lo que nos permite determinar el tipo de relación adecuado para cada par de clases.

Relación Mascota – DispositivoLocalizacion

La relación entre estas dos clases es de composición. Esto se concluye gracias al punto 5 de la consigna que es determinante: "El dispositivo no puede existir de forma independiente". Esto significa que:

- El dispositivo solo cobra sentido cuando está asociado a una mascota
- Su ciclo de vida está ligado al de la mascota
- No se puede crear un dispositivo sin una mascota que lo posea

Estas características corresponden exactamente a una relación de composición, donde el objeto parte (dispositivo) no puede existir sin el objeto todo (mascota)

Características de la composición implementada en el código:

Característica	Implementación
El dispositivo no existe sin mascota	El constructor de DispositivoLocalizacion requiere una Mascota
La mascota crea el dispositivo	El método crearDispositivo() de Mascota instancia el dispositivo
Unicidad	La mascota solo puede tener un dispositivo (validado en crearDispositivo())
Bidireccionalidad	Ambas clases se conocen mutuamente (referencia cruzada)

Esta implementación trae los siguientes beneficios:

- Encapsulamiento: La lógica de creación del dispositivo está dentro de la mascota
- Consistencia: Se garantiza que nunca exista un dispositivo huérfano

Relación DispositivoLocalizacion – RegistroUbicacion

La relación entre estas dos clases es de composición, esto gracias a los puntos 3 y 4 de la consigna que establecen:

- Un dispositivo puede estar asociado con cero o más registros de ubicación.
- Cada registro de ubicación pertenece a un único dispositivo.

Además de que los registros son generados por el dispositivo en momentos específicos, un registro no tiene sentido sin el dispositivo que lo generó y el dispositivo es el dueño de sus registros

Estas características también corresponden a una relación de composición, donde el dispositivo es el contenedor y los registros son las partes contenidas.

Características de la composición implementada en el código:

Característica	Implementación
El registro no existe sin dispositivo	El constructor de RegistroUbicacion requiere un DispositivoLocalizacion
El dispositivo genera los registros	Método registrarUbicacion() en DispositivoLocalizacion
Pertenencia	Cada registro conoce a su dispositivo (referencia)

Con esta implementación logramos que la lógica de registro está donde corresponde (en el dispositivo), se garantiza que cada registro tenga un dispositivo padre y lo que los registros mantienen referencia a su dispositivo original.

Relaciones de dependencia

Además de las relaciones estructurales (composición), existen relaciones de dependencia donde una clase utiliza a otra para cumplir sus funciones.

Dependencias identificadas:

Dependencia	Justificación
Main - Menu	Main crea una instancia de Menu y llama a su método mostrar() para iniciar la aplicación
Menu - Mascota	Menu utiliza objetos Mascota para registrarlas, buscarlas y mostrarlas
Menu - DispositivoLocalizacion	Menu accede a los dispositivos a través de las mascotas para registrar ubicaciones
Menu - RegistroUbicacion	Menu muestra los registros utilizando su método toString()